arXiv:1602.05629v4 [cs.LG] 26 Jan 2023

Communication-Efficient Learning of Deep Networks from Decentralized Data

H. Brendan McMahan **Eider Moore** **Daniel Ramage** **Seth Hampson** **Blaise Agüera y Arcas**
Google, Inc., 651 N 34th St., Seattle, WA 98103 USA

Abstract

Modern mobile devices have access to a wealth of data suitable for learning models, which in turn can greatly improve the user experience on the device. For example, language models can improve speech recognition and text entry, and image models can automatically select good photos. However, this rich data is often privacy sensitive, large in quantity, or both, which may preclude logging to the data center and training there using conventional approaches. We advocate an alternative that leaves the training data distributed on the mobile devices, and learns a shared model by aggregating locally-computed updates. We term this decentralized approach *Federated Learning*.

We present a practical method for the federated learning of deep networks based on iterative model averaging, and conduct an extensive empirical evaluation, considering five different model architectures and four datasets. These experiments demonstrate the approach is robust to the unbalanced and non-IID data distributions that are a defining characteristic of this setting. Communication costs are the principal constraint, and we show a reduction in required communication rounds by 10–100× as compared to synchronized stochastic gradient descent.

1 Introduction

Increasingly, phones and tablets are the primary computing devices for many people [30, 2]. The powerful sensors on these devices (including cameras, microphones, and GPS), combined with the fact they are frequently carried, means they have access to an unprecedented amount of data, much of it private in nature. Models learned on such data hold the promise of greatly improving usability by powering more intelligent applications, but the sensitive nature of the data means there are risks and responsibilities to storing it in a centralized location.

We investigate a learning technique that allows users to collectively reap the benefits of shared models trained from this rich data, without the need to centrally store it. We term our approach *Federated Learning*, since the learning task is solved by a loose federation of participating devices (which we refer to as *clients*) which are coordinated by a central *server*. Each client has a local training dataset which is never uploaded to the server. Instead, each client computes an update to the current global model maintained by the server, and only this update is communicated. This is a direct application of the principle of *focused collection* or *data minimization* proposed by the 2012 White House report on privacy of consumer data [39]. Since these updates are specific to improving the current model, there is no reason to store them once they have been applied.

A principal advantage of this approach is the decoupling of model training from the need for direct access to the raw training data. Clearly, some trust of the server coordinating the training is still required. However, for applications where the training objective can be specified on the basis of data available on each client, federated learning can significantly reduce privacy and security risks by limiting the attack surface to only the device, rather than the device and the cloud.

Our primary contributions are 1) the identification of the problem of training on decentralized data from mobile devices as an important research direction; 2) the selection of a straightforward and practical algorithm that can be applied to this setting; and 3) an extensive empirical evaluation of the proposed approach. More concretely, we introduce the `FederatedAveraging` algorithm, which combines local stochastic gradient descent (SGD) on each client with a server that performs model averaging. We perform extensive experiments on this algorithm, demonstrating it is robust to unbalanced and non-IID data distributions, and can reduce the rounds of communication needed to train a deep network on decentralized data by orders of magnitude.

Appearing in Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS) 2017, Fort Lauderdale, Flordia, USA. JMLR: W&CP volume 54. Copyright 2017 by the authors.

Federated Learning Ideal problems for federated learning have the following properties: 1) Training on real-world data from mobile devices provides a distinct advantage over training on proxy data that is generally available in the data center. 2) This data is privacy sensitive or large in size (compared to the size of the model), so it is preferable not to log it to the data center purely for the purpose of model training (in service of the *focused collection* principle). 3) For supervised tasks, labels on the data can be inferred naturally from user interaction.

Many models that power intelligent behavior on mobile devices fit the above criteria. As two examples, we consider *image classification*, for example predicting which photos are most likely to be viewed multiple times in the future, or shared; and *language models*, which can be used to improve voice recognition and text entry on touch-screen keyboards by improving decoding, next-word-prediction, and even predicting whole replies [10]. The potential training data for both these tasks (all the photos a user takes and everything they type on their mobile keyboard, including passwords, URLs, messages, etc.) can be privacy sensitive. The distributions from which these examples are drawn are also likely to differ substantially from easily available proxy datasets: the use of language in chat and text messages is generally much different than standard language corpora, e.g., Wikipedia and other web documents; the photos people take on their phone are likely quite different than typical Flickr photos. And finally, the labels for these problems are directly available: entered text is self-labeled for learning a language model, and photo labels can be defined by natural user interaction with their photo app (which photos are deleted, shared, or viewed).

Both of these tasks are well-suited to learning a neural network. For image classification feed-forward deep networks, and in particular convolutional networks, are well-known to provide state-of-the-art results [26, 25]. For language modeling tasks recurrent neural networks, and in particular LSTMs, have achieved state-of-the-art results [20, 5, 22].

Privacy Federated learning has distinct privacy advantages compared to data center training on persisted data. Holding even an "anonymized" dataset can still put user privacy at risk via joins with other data [37]. In contrast, the information transmitted for federated learning is the minimal update necessary to improve a particular model (naturally, the strength of the privacy benefit depends on the content of the updates).[1] The updates themselves can (and should) be ephemeral. They will never contain more information than the raw training data (by the data processing inequality), and will generally contain much less. Further, the source of the updates is not needed by the aggregation algorithm, so updates can be transmitted without identifying meta-data over a mix network such as Tor [7] or via a trusted third party. We briefly discuss the possibility of combining federated learning with secure multiparty computation and differential privacy at the end of the paper.

[1]For example, if the update is the total gradient of the loss on all of the local data, and the features are a sparse bag-of-words, then the non-zero gradients reveal exactly which words the user has entered on the device. In contrast, the sum of many gradients for a dense model such as a CNN offers a harder target for attackers seeking information about individual training instances (though attacks are still possible).

Federated Optimization We refer to the optimization problem implicit in federated learning as federated optimization, drawing a connection (and contrast) to distributed optimization. Federated optimization has several key properties that differentiate it from a typical distributed optimization problem:

- **Non-IID** The training data on a given client is typically based on the usage of the mobile device by a particular user, and hence any particular user's local dataset will not be representative of the population distribution.
- **Unbalanced** Similarly, some users will make much heavier use of the service or app than others, leading to varying amounts of local training data.
- **Massively distributed** We expect the number of clients participating in an optimization to be much larger than the average number of examples per client.
- **Limited communication** Mobile devices are frequently offline or on slow or expensive connections.

In this work, our emphasis is on the non-IID and unbalanced properties of the optimization, as well as the critical nature of the communication constraints. A deployed federated optimization system must also address a myriad of practical issues: client datasets that change as data is added and deleted; client availability that correlates with the local data distribution in complex ways (e.g., phones from speakers of American English will likely be plugged in at different times than speakers of British English); and clients that never respond or send corrupted updates.

These issues are beyond the scope of the current work; instead, we use a controlled environment that is suitable for experiments, but still addresses the key issues of client availability and unbalanced and non-IID data. We assume a synchronous update scheme that proceeds in rounds of communication. There is a fixed set of K clients, each with a fixed local dataset. At the beginning of each round, a random fraction C of clients is selected, and the server sends the current global algorithm state to each of these clients (e.g., the current model parameters). We only select a fraction of clients for efficiency, as our experiments show diminishing returns for adding more clients beyond a certain point. Each selected client then performs local computation based on the global state and its local dataset, and sends an update to the server. The server then applies these updates to its global state, and the process repeats.

While we focus on non-convex neural network objectives, the algorithm we consider is applicable to any finite-sum objective of the form

$$\min_{w \in \mathbb{R}^d} f(w) \qquad \text{where} \qquad f(w) \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^{n} f_i(w). \quad (1)$$

For a machine learning problem, we typically take $f_i(w) = \ell(x_i, y_i; w)$, that is, the loss of the prediction on example (x_i, y_i) made with model parameters w. We assume there are K clients over which the data is partitioned, with $\mathcal{P}_k$ the set of indexes of data points on client k, with $n_k = |\mathcal{P}_k|$. Thus, we can re-write the objective (1) as

$$f(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w) \quad \text{where} \quad F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} f_i(w).$$

If the partition $\mathcal{P}_k$ was formed by distributing the training examples over the clients uniformly at random, then we would have $\mathbb{E}_{\mathcal{P}_k}[F_k(w)] = f(w)$, where the expectation is over the set of examples assigned to a fixed client k. This is the IID assumption typically made by distributed optimization algorithms; we refer to the case where this does not hold (that is, F_k could be an arbitrarily bad approximation to f) as the non-IID setting.

In data center optimization, communication costs are relatively small, and computational costs dominate, with much of the recent emphasis being on using GPUs to lower these costs. In contrast, in federated optimization communication costs dominate — we will typically be limited by an upload bandwidth of 1 MB/s or less. Further, clients will typically only volunteer to participate in the optimization when they are charged, plugged-in, and on an unmetered wi-fi connection. Further, we expect each client will only participate in a small number of update rounds per day. On the other hand, since any single on-device dataset is small compared to the total dataset size, and modern smartphones have relatively fast processors (including GPUs), computation becomes essentially free compared to communication costs for many model types. Thus, our goal is to use additional computation in order to decrease the number of rounds of communication needed to train a model. There are two primary ways we can add computation: 1) *increased parallelism*, where we use more clients working independently between each communication round; and, 2) *increased computation on each client*, where rather than performing a simple computation like a gradient calculation, each client performs a more complex calculation between each communication round. We investigate both of these approaches, but the speedups we achieve are due primarily to adding more computation on each client, once a minimum level of parallelism over clients is used.

Related Work Distributed training by iteratively averaging locally trained models has been studied by McDonald et al. [28] for the perceptron and Povey et al. [31] for speech recognition DNNs. Zhang et al. [42] studies an asynchronous approach with "soft" averaging. These works only consider the cluster / data center setting (at most 16 workers, wall-clock time based on fast networks), and do not consider datasets that are unbalanced and non-IID, properties that are essential to the federated learning setting. We adapt this style of algorithm to the federated setting and perform the appropriate empirical evaluation, which asks different questions than those relevant in the data center setting, and requires different methodology.

Using similar motivation to ours, Neverova et al. [29] also discusses the advantages of keeping sensitive user data on device. The work of Shokri and Shmatikov [35] is related in several ways: they focus on training deep networks, emphasize the importance of privacy, and address communication costs by only sharing a subset of the parameters during each round of communication; however, they also do not consider unbalanced and non-IID data, and the empirical evaluation is limited.

In the convex setting, the problem of distributed optimization and estimation has received significant attention [4, 15, 33], and some algorithms do focus specifically on communication efficiency [45, 34, 40, 27, 43]. In addition to assuming convexity, this existing work generally requires that the number of clients is much smaller than the number of examples per client, that the data is distributed across the clients in IID fashion, and that each node has an identical number of data points — all of these assumptions are violated in the federated optimization setting. Asynchronous distributed forms of SGD have also been applied to training neural networks, e.g., Dean et al. [12], but these approaches require a prohibitive number of updates in the federated setting. Distributed consensus algorithms (e.g., [41]) relax the IID assumption, but are still not a good fit for communication-constrained optimization over very many clients.

One endpoint of the (parameterized) algorithm family we consider is simple one-shot averaging, where each client solves for the model that minimizes (possibly regularized) loss on their local data, and these models are averaged to produce the final global model. This approach has been studied extensively in the convex case with IID data, and it is known that in the worst-case, the global model produced is no better than training a model on a single client [44, 3, 46].

2 The `FederatedAveraging` Algorithm

The recent multitude of successful applications of deep learning have almost exclusively relied on variants of stochastic gradient descent (SGD) for optimization; in fact, many advances can be understood as adapting the structure of the model (and hence the loss function) to be more amenable to optimization by simple gradient-based methods [16]. Thus, it is natural that we build algorithms for

federated optimization by starting from SGD.

SGD can be applied naively to the federated optimization problem, where a single batch gradient calculation (say on a randomly selected client) is done per round of communication. This approach is computationally efficient, but requires very large numbers of rounds of training to produce good models (e.g., even using an advanced approach like batch normalization, Ioffe and Szegedy [21] trained MNIST for 50000 steps on minibatches of size 60). We consider this baseline in our CIFAR-10 experiments.

In the federated setting, there is little cost in wall-clock time to involving more clients, and so for our baseline we use large-batch synchronous SGD; experiments by Chen et al. [8] show this approach is state-of-the-art in the data center setting, where it outperforms asynchronous approaches. To apply this approach in the federated setting, we select a C-fraction of clients on each round, and compute the gradient of the loss over all the data held by these clients. Thus, C controls the *global* batch size, with $C = 1$ corresponding to full-batch (non-stochastic) gradient descent.[2] We refer to this baseline algorithm as `FederatedSGD` (or `FedSGD`).

A typical implementation of `FedSGD` with $C = 1$ and a fixed learning rate η has each client k compute $g_k = \nabla F_k(w_t)$, the average gradient on its local data at the current model w_t, and the central server aggregates these gradients and applies the update $w_{t+1} \leftarrow w_t - \eta \sum_{k=1}^{K} \frac{n_k}{n} g_k$, since $\sum_{k=1}^{K} \frac{n_k}{n} g_k = \nabla f(w_t)$. An equivalent update is given by $\forall k,\ w_{t+1}^k \leftarrow w_t - \eta g_k$ and then $w_{t+1} \leftarrow \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k$. That is, each client locally takes one step of gradient descent on the current model using its local data, and the server then takes a weighted average of the resulting models. Once the algorithm is written this way, we can add more computation to each client by iterating the local update $w^k \leftarrow w^k - \eta \nabla F_k(w^k)$ multiple times before the averaging step. We term this approach `FederatedAveraging` (or `FedAvg`). The amount of computation is controlled by three key parameters: C, the fraction of clients that perform computation on each round; E, then number of training passes each client makes over its local dataset on each round; and B, the local minibatch size used for the client updates. We write $B = \infty$ to indicate that the full local dataset is treated as a single minibatch. Thus, at one endpoint of this algorithm family, we can take $B = \infty$ and $E = 1$ which corresponds exactly to `FedSGD`. For a client with n_k local examples, the number of local updates per round is given by $u_k = E\frac{n_k}{B}$; Complete pseudo-code is given in Algorithm 1.

For general non-convex objectives, averaging models in parameter space could produce an arbitrarily bad model.

[2]While the batch selection mechanism is different than selecting a batch by choosing individual examples uniformly at random, the batch gradients g computed by `FedSGD` still satisfy $\mathbb{E}[g] = \nabla f(w)$.

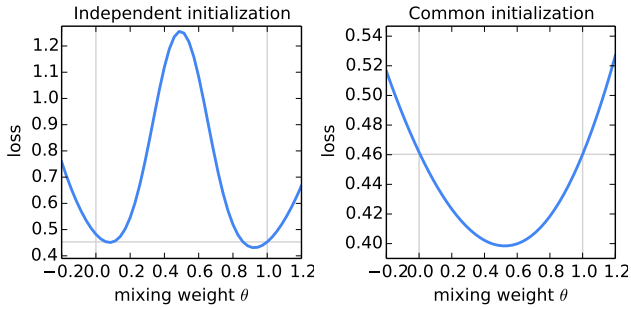

Figure 1: The loss on the full MNIST training set for models generated by averaging the parameters of two models w and w' using $\theta w + (1-\theta)w'$ for 50 evenly spaced values $\theta \in [-0.2, 1.2]$.The models w and w' were trained using SGD on different small datasets. For the left plot, w and w' were initialized using different random seeds; for the right plot, a shared seed was used. Note the different y-axis scales. The horizontal line gives the best loss achieved by w or w' (which were quite close, corresponding to the vertical lines at $\theta = 0$ and $\theta = 1$). With shared initialization, averaging the models produces a significant reduction in the loss on the total training set (much better than the loss of either parent model).

Following the approach of Goodfellow et al. [17], we see exactly this bad behavior when we average two MNIST digit-recognition models[3] trained from different initial conditions (Figure 1, left). For this figure, the parent models w and w' were each trained on non-overlapping IID samples of 600 examples from the MNIST training set. Training was via SGD with a fixed learning rate of 0.1 for 240 updates on minibatches of size 50 (or $E = 20$ passes over the mini-datasets of size 600). This is approximately the amount of training where the models begin to overfit their local datasets.

Recent work indicates that in practice, the loss surfaces of sufficiently over-parameterized NNs are surprisingly well-behaved and in particular less prone to bad local minima than previously thought [11, 17, 9]. And indeed, when we start two models *from the same random initialization* and then again train each independently on a different subset of the data (as described above), we find that naive parameter averaging works surprisingly well (Figure 1, right): the average of these two models, $\frac{1}{2}w + \frac{1}{2}w'$, achieves significantly lower loss on the full MNIST training set than the best model achieved by training on either of the small datasets independently. While Figure 1 starts from a random initialization, note a shared starting model w_t is used for each round of `FedAvg`, and so the same intuition applies.

[3]We use the "2NN" multi-layer perceptron described in Section 3.

Algorithm 1 `FederatedAveraging`. The K clients are indexed by k; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:
  initialize w_0
  for each round $t = 1, 2, \ldots$ **do**
    $m \leftarrow \max(C \cdot K, 1)$
    $S_t \leftarrow$ (random set of m clients)
    for each client $k \in S_t$ **in parallel do**
      $w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$
    $m_t \leftarrow \sum_{k \in S_t} n_k$
    $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k$ *// Erratum*[4]

ClientUpdate(k, w)**:** *// Run on client* k
  $\mathcal{B} \leftarrow$ (split $\mathcal{P}_k$ into batches of size B)
  for each local epoch i from 1 to E **do**
    for batch $b \in \mathcal{B}$ **do**
      $w \leftarrow w - \eta \nabla \ell(w; b)$
  return w to server

3 Experimental Results

We are motivated by both image classification and language modeling tasks where good models can greatly enhance the usability of mobile devices. For each of these tasks we first picked a proxy dataset of modest enough size that we could thoroughly investigate the hyperparameters of the `FedAvg` algorithm. While each individual training run is relatively small, we trained over 2000 individual models for these experiments. We then present results on the benchmark CIFAR-10 image classification task. Finally, to demonstrate the effectiveness of `FedAvg` on a real-world problem with a natural partitioning of the data over clients, we evaluate on a large language modeling task.

Our initial study includes three model families on two datasets. The first two are for the MNIST digit recognition task [26]: 1) A simple multilayer-perceptron with 2-hidden layers with 200 units each using ReLu activations (199,210 total parameters), which we refer to as the MNIST 2NN. 2) A CNN with two 5x5 convolution layers (the first with 32 channels, the second with 64, each followed with 2x2 max pooling), a fully connected layer with 512 units and ReLu activation, and a final softmax output layer (1,663,370 total parameters). To study federated optimization, we also need to specify how the data is distributed over the clients. We study two ways of partitioning the MNIST data over clients: **IID**, where the data is shuffled, and then partitioned into 100 clients each receiving 600 examples, and **Non-IID**, where we first *sort* the data by digit label, divide it into 200 shards of size 300, and assign each of 100 clients 2 shards. This is a pathological non-IID partition of the data, as most clients will only have examples of two digits, letting us explore the degree to which our algorithms will break on highly non-IID data. Both of these partitions are balanced, however.[5]

For language modeling, we built a dataset from *The Complete Works of William Shakespeare* [32]. We construct a client dataset for each speaking role in each play with at least two lines. This produced a dataset with 1146 clients. For each client, we split the data into a set of training lines (the first 80% of lines for the role), and test lines (the last 20%, rounded up to at least one line). The resulting dataset has 3,564,579 characters in the training set, and 870,014 characters[6] in the test set. This data is substantially unbalanced, with many roles having only a few lines, and a few with a large number of lines. Further, observe the test set is not a random sample of lines, but is temporally separated by the chronology of each play. Using an identical train/test split, we also form a balanced and IID version of the dataset, also with 1146 clients.

On this data we train a stacked character-level LSTM language model, which after reading each character in a line, predicts the next character [22]. The model takes a series of characters as input and embeds each of these into a learned 8 dimensional space. The embedded characters are then processed through 2 LSTM layers, each with 256 nodes. Finally the output of the second LSTM layer is sent to a softmax output layer with one node per character. The full model has 866,578 parameters, and we trained using an unroll length of 80 characters.

SGD is sensitive to the tuning of the learning-rate parameter η. The results reported here are based on training over a sufficiently wide grid of learning rates (typically 11-13 values for η on a multiplicative grid of resolution $10^{\frac{1}{3}}$ or $10^{\frac{1}{6}}$). We checked to ensure the best learning rates were in the middle of our grids, and that there was not a significant difference between the best learning rates. Unless otherwise noted, we plot metrics for the best performing rate selected individually for each x-axis value. We find that the optimal learning rates do not vary too much as a function of the other parameters.

Increasing parallelism We first experiment with the client fraction C, which controls the amount of multi-client parallelism. Table 1 shows the impact of varying C for both MNIST models. We report the number of communication rounds necessary to achieve a target test-set accuracy. To compute this, we construct a learning curve for each combination of parameter settings, optimizing η as described above and then making each curve monotonically improving by taking the best value of test-set accuracy achieved over

[4] Earlier versions of this paper incorrectly indicated summation over all K clients here.

[5] We performed additional experiments on unbalanced versions of these datasets, and found them to in fact be slightly easier for `FedAvg`.

[6] We always use character to refer to a one byte string, and use role to refer to a part in the play.

Table 1: Effect of the client fraction C on the MNIST 2NN with $E = 1$ and CNN with $E = 5$. Note $C = 0.0$ corresponds to one client per round; since we use 100 clients for the MNIST data, the rows correspond to 1, 10 20, 50, and 100 clients. Each table entry gives the number of rounds of communication necessary to achieve a test-set accuracy of 97% for the 2NN and 99% for the CNN, along with the speedup relative to the $C = 0$ baseline. Five runs with the large batch size did not reach the target accuracy in the allowed time.

2NN	IID		NON-IID	
C	$B = \infty$	$B = 10$	$B = \infty$	$B = 10$
0.0	1455	316	4278	3275
0.1	1474 (1.0×)	87 (3.6×)	1796 (2.4×)	664 (4.9×)
0.2	1658 (0.9×)	77 (4.1×)	1528 (2.8×)	619 (5.3×)
0.5	— (—)	75 (4.2×)	— (—)	443 (7.4×)
1.0	— (—)	70 (4.5×)	— (—)	380 (8.6×)
CNN, $E = 5$				
0.0	387	50	1181	956
0.1	339 (1.1×)	18 (2.8×)	1100 (1.1×)	206 (4.6×)
0.2	337 (1.1×)	18 (2.8×)	978 (1.2×)	200 (4.8×)
0.5	164 (2.4×)	18 (2.8×)	1067 (1.1×)	261 (3.7×)
1.0	246 (1.6×)	16 (3.1×)	— (—)	97 (9.9×)

all prior rounds. We then calculate the number of rounds where the curve crosses the target accuracy, using linear interpolation between the discrete points forming the curve. This is perhaps best understood by reference to Figure 2, where the gray lines show the targets.

With $B = \infty$ (for MNIST processing all 600 client examples as a single batch per round), there is only a small advantage in increasing the client fraction. Using the smaller batch size $B = 10$ shows a significant improvement in using $C \geq 0.1$, especially in the non-IID case. Based on these results, for most of the remainder of our experiments we fix $C = 0.1$, which strikes a good balance between computational efficiency and convergence rate. Comparing the number of rounds for the $B = \infty$ and $B = 10$ columns in Table 1 shows a dramatic speedup, which we investigate next.

Increasing computation per client In this section, we fix $C = 0.1$, and add more computation per client on each round, either decreasing B, increasing E, or both. Figure 2 demonstrates that adding more local SGD updates per round can produce a dramatic decrease in communication costs, and Table 2 quantifies these speedups. The expected number of updates per client per round is $u = (\mathbb{E}[n_k]/B)E = nE/(KB)$, where the expectation is over the draw of a random client k. We order the rows in each section of Table 2 by this statistic. We see that increasing u by varying both E and B is effective. As long as B is large enough to take full advantage of available parallelism on the client hardware, there is essentially no cost in computation time for lowering it, and so in practice this should be the first parameter tuned.

Table 2: Number of communication rounds to reach a target accuracy for `FedAvg`, versus `FedSGD` (first row, $E = 1$ and $B = \infty$). The u column gives $u = En/(KB)$, the expected number of updates per round.

MNIST CNN, 99% ACCURACY					
CNN	E	B	u	IID	NON-IID
FEDSGD	1	∞	1	626	483
FEDAVG	5	∞	5	179 (3.5×)	1000 (0.5×)
FEDAVG	1	50	12	65 (9.6×)	600 (0.8×)
FEDAVG	20	∞	20	234 (2.7×)	672 (0.7×)
FEDAVG	1	10	60	34 (18.4×)	350 (1.4×)
FEDAVG	5	50	60	29 (21.6×)	334 (1.4×)
FEDAVG	20	50	240	32 (19.6×)	426 (1.1×)
FEDAVG	5	10	300	20 (31.3×)	229 (2.1×)
FEDAVG	20	10	1200	18 (34.8×)	173 (2.8×)
SHAKESPEARE LSTM, 54% ACCURACY					
LSTM	E	B	u	IID	NON-IID
FEDSGD	1	∞	1.0	2488	3906
FEDAVG	1	50	1.5	1635 (1.5×)	549 (7.1×)
FEDAVG	5	∞	5.0	613 (4.1×)	597 (6.5×)
FEDAVG	1	10	7.4	460 (5.4×)	164 (23.8×)
FEDAVG	5	50	7.4	401 (6.2×)	152 (25.7×)
FEDAVG	5	10	37.1	192 (13.0×)	41 (95.3×)

For the IID partition of the MNIST data, using more computation per client decreases the number of rounds to reach the target accuracy by $35\times$ for the CNN and $46\times$ for the 2NN (see Table 4 in Appendix A for details for the 2NN). The speedups for the pathologically partitioned non-IID data are smaller, but still substantial ($2.8 - 3.7\times$). It is impressive that averaging provides *any* advantage (vs. actually diverging) when we naively average the parameters of models trained on entirely different pairs of digits. Thus, we view this as strong evidence for the robustness of this approach.

The unbalanced and non-IID distribution of the Shakespeare (by role in the play) is much more representative of the kind of data distribution we expect for real-world applications. Encouragingly, for this problem learning on the non-IID and unbalanced data is actually much easier (a $95\times$ speedup vs $13\times$ for the balanced IID data); we conjecture this is largely due to the fact some roles have relatively large local datasets, which makes increased local training particularly valuable.

For all three model classes, `FedAvg` converges to a higher level of test-set accuracy than the baseline `FedSGD` models. This trend continues even if the lines are extended beyond the plotted ranges. For example, for the CNN the $B = \infty, E = 1$ `FedSGD` model eventually reaches 99.22% accuracy after 1200 rounds (and had not improved further after 6000 rounds), while the $B = 10, E = 20$ `FedAvg` model reaches an accuracy of 99.44% after 300 rounds. We conjecture that in addition to lowering communication costs, model averaging produces a regularization benefit similar to that achieved by dropout [36].

We are primarily concerned with generalization performance, but `FedAvg` is effective at optimizing the training loss as well, even beyond the point where test-set accuracy plateaus. We observed similar behavior for all three model classes, and present plots for the MNIST CNN in Figure 6

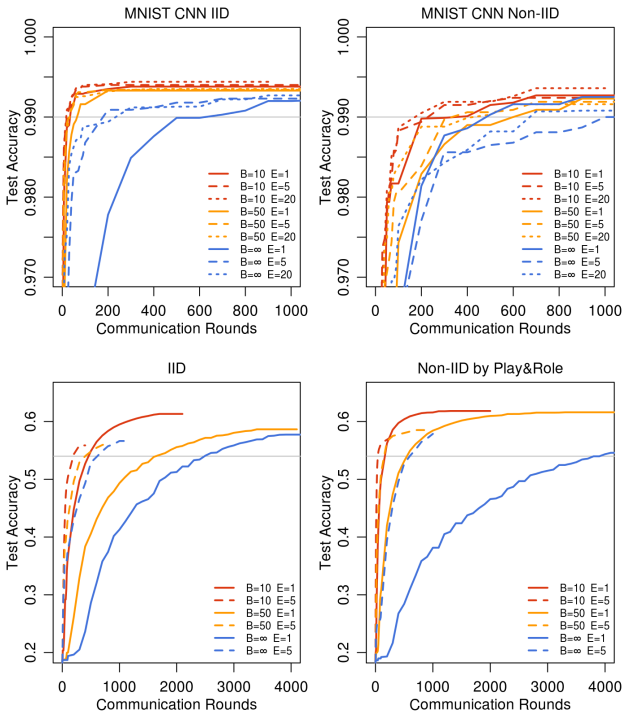

Figure 2: Test set accuracy vs. communication rounds for the MNIST CNN (IID and then pathological non-IID) and Shakespeare LSTM (IID and then by Play&Role) with $C = 0.1$ and optimized η. The gray lines show the target accuracies used in Table 2. Plots for the 2NN are given as Figure 7 in Appendix A.

in Appendix A.

Can we over-optimize on the client datasets? The current model parameters only influence the optimization performed in each `ClientUpdate` via initialization. Thus, as $E \to \infty$, at least for a convex problem eventually the initial conditions should be irrelevant, and the global minimum would be reached regardless of initialization. Even for a non-convex problem, one might conjecture the algorithm would converge to the same local minimum as long as the initialization was in the same basin. That is, we would expect that while one round of averaging might produce a reasonable model, additional rounds of communication (and averaging) would not produce further improvements.

Figure 3 shows the impact of large E during initial training on the Shakespeare LSTM problem. Indeed, for very large numbers of local epochs, `FedAvg` can plateau or diverge.[7] This result suggests that for some models, especially in the later stages of convergence, it may be useful to decay the

[7]Note that due to this behavior and because for large E not all experiments for all learning rates were run for the full number of rounds, we report results for a fixed learning rate (which perhaps surprisingly was near-optimal across the range of E parameters) and without forcing the lines to be monotonic.

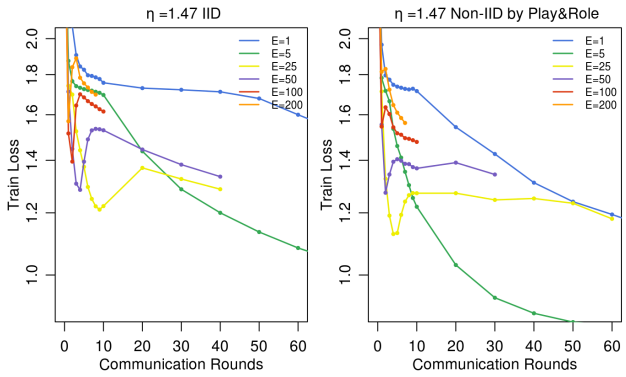

Figure 3: The effect of training for many local epochs (large E) between averaging steps, fixing $B = 10$ and $C = 0.1$ for the Shakespeare LSTM with a fixed learning rate $\eta = 1.47$.

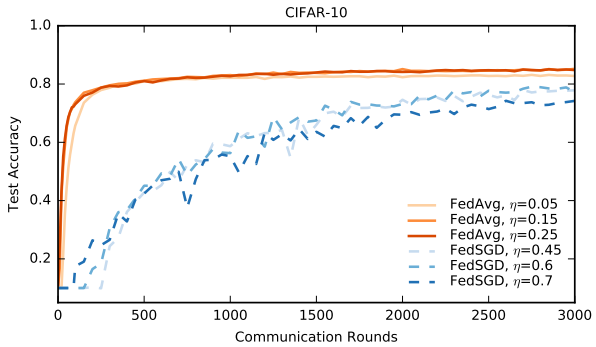

Figure 4: Test accuracy versus communication for the CIFAR10 experiments. `FedSGD` uses a learning-rate decay of 0.9934 per round; `FedAvg` uses $B = 50$, learning-rate decay of 0.99 per round, and $E = 5$.

amount of local computation per round (moving to smaller E or larger B) in the same way decaying learning rates can be useful. Figure 8 in Appendix A gives the analogous experiment for the MNIST CNN. Interestingly, for this model we see no significant degradation in the convergence rate for large values of E. However, we see slightly better performance for $E = 1$ versus $E = 5$ for the large-scale language modeling task described below (see Figure 10 in Appendix A).

CIFAR experiments We also ran experiments on the CIFAR-10 dataset [24] to further validate `FedAvg`. The dataset consists of 10 classes of 32x32 images with three RGB channels. There are 50,000 training examples and 10,000 testing examples, which we partitioned into 100 clients each containing 500 training and 100 testing examples; since there isn't a natural user partitioning of this data, we considered the balanced and IID setting. The model architecture was taken from the TensorFlow tutorial [38], which consists of two convolutional layers followed by two fully connected layers and then a linear transformation layer

Table 3: Number of rounds and speedup relative to baseline SGD to reach a target test-set accuracy on CIFAR10. SGD used a minibatch size of 100. FedSGD and FedAvg used $C = 0.1$, with FedAvg using $E = 5$ and $B = 50$.

ACC.	80%	82%	85%
SGD	18000 (—)	31000 (—)	99000 (—)
FEDSGD	3750 (4.8×)	6600 (4.7×)	N/A (—)
FEDAVG	280 (64.3×)	630 (49.2×)	2000 (49.5×)

to produce logits, for a total of about 10^6 parameters. Note that state-of-the-art approaches have achieved a test accuracy of 96.5% [19] for CIFAR; nevertheless, the standard model we use is sufficient for our needs, as our goal is to evaluate our optimization method, not achieve the best possible accuracy on this task. The images are preprocessed as part of the training input pipeline, which consists of cropping the images to 24x24, randomly flipping left-right and adjusting the contrast, brightness and whitening.

For these experiments, we considered an additional baseline, standard SGD training on the full training set (no user partitioning), using minibatches of size 100. We achieved an 86% test accuracy after 197,500 minibatch updates (each minibatch update requires a communication round in the federated setting). FedAvg achieves a similar test accuracy of 85% after only 2,000 communication rounds. For all algorithms, we tuned a learning-rate decay parameter in addition to the initial learning rate. Table 3 gives the number of communication rounds for baseline SGD, FedSGD, and FedAvg to reach three different accuracy targets, and Figure 4 gives learning-rate curves for FedAvg versus FedSGD.

By running experiments with minibatches of size $B = 50$ for both SGD and FedAvg, we can also look at accuracy as a function of the number of such minibatch gradient calculations. We expect SGD to do better here, because a sequential step is taken after each minibatch computation. However, as Figure 9 in the appendix shows, for modest values of C and E, FedAvg makes a similar amount of progress per minibatch computation. Further, we see that both standard SGD and FedAvg with only one client per round ($C = 0$), demonstrate significant oscillations in accuracy, whereas averaging over more clients smooths this out.

Large-scale LSTM experiments We ran experiments on a large-scale next-word prediction task to demonstrate the effectiveness of our approach on a real-world problem. Our training dataset consists 10 million public posts from a large social network. We grouped the posts by author, for a total of over 500,000 clients. This dataset is a realistic proxy for the type of text entry data that would be present on a user's mobile device. We limited each client dataset to at most 5000 words, and report accuracy (the fraction of the data where the highest predicted probability was on the correct next word, out of 10000 possibilities) on a test set of $1e5$

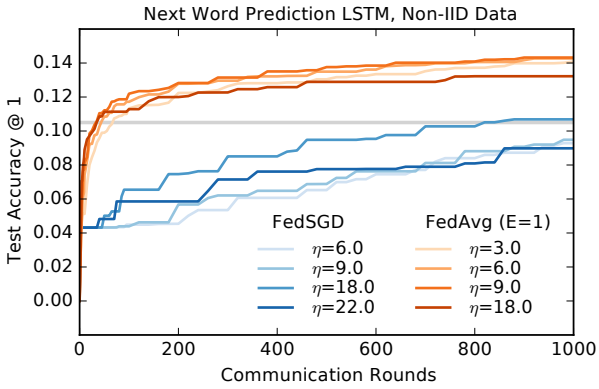

Figure 5: Monotonic learning curves for the large-scale language model word LSTM.

posts from different (non-training) authors. Our model is a 256 node LSTM on a vocabulary of 10,000 words. The input and output embeddings for each word were of dimension 192, and co-trained with the model; there are 4,950,544 parameters in all. We used an unroll of 10 words.

These experiments required significant computational resources and so we did not explore hyper-parameters as thoroughly: all runs trained on 200 clients per round; FedAvg used $B = 8$ and $E = 1$. We explored a variety of learning rates for FedAvg and the baseline FedSGD. Figure 5 shows monotonic learning curves for the best learning rates. FedSGD with $\eta = 18.0$ required 820 rounds to reach 10.5% accuracy, while FedAvg with $\eta = 9.0$ reached an accuracy of 10.5% in only 35 communication rounds (23× fewer then FedSGD). We observed lower variance in test accuracy for FedAvg, see Figure 10 in Appendix A. This figure also include results for $E = 5$, which performed slightly worse than $E = 1$.

4 Conclusions and Future Work

Our experiments show that federated learning can be made practical, as FedAvg trains high-quality models using relatively few rounds of communication, as demonstrated by results on a variety of model architectures: a multi-layer perceptron, two different convolutional NNs, a two-layer character LSTM, and a large-scale word-level LSTM.

While federated learning offers many practical privacy benefits, providing stronger guarantees via differential privacy [14, 13, 1], secure multi-party computation [18], or their combination is an interesting direction for future work. Note that both classes of techniques apply most naturally to synchronous algorithms like FedAvg.[8]

[8] Subsequent to this work, Bonawitz et al. [6] introduced an efficient secure aggregation protocol for federated learning, and Konečný et al. [23] presented algorithms for further decreasing communication costs.

References

[1] Martin Abadi, Andy Chu, Ian Goodfellow, Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *23rd ACM Conference on Computer and Communications Security (ACM CCS)*, 2016.

[2] Monica Anderson. Technology device ownership: 2015. `http://www.pewinternet.org/2015/10/29/technology-device-ownership-2015/`, 2015.

[3] Yossi Arjevani and Ohad Shamir. Communication complexity of distributed convex learning and optimization. In *Advances in Neural Information Processing Systems 28*. 2015.

[4] Maria-Florina Balcan, Avrim Blum, Shai Fine, and Yishay Mansour. Distributed learning, communication complexity and privacy. *arXiv preprint arXiv:1204.3514*, 2012.

[5] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Janvin. A neural probabilistic language model. *J. Mach. Learn. Res.*, 2003.

[6] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for federated learning on user-held data. In *NIPS Workshop on Private Multi-Party Machine Learning*, 2016.

[7] David L. Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Commun. ACM*, 24(2), 1981.

[8] Jianmin Chen, Rajat Monga, Samy Bengio, and Rafal Jozefowicz. Revisiting distributed synchronous sgd. In *ICLR Workshop Track*, 2016.

[9] Anna Choromanska, Mikael Henaff, Michaël Mathieu, Gérard Ben Arous, and Yann LeCun. The loss surfaces of multilayer networks. In *AISTATS*, 2015.

[10] Greg Corrado. Computer, respond to this email. `http://googleresearch.blogspot.com/2015/11/computer-respond-to-this-email.html`, November 2015.

[11] Yann N. Dauphin, Razvan Pascanu, Çaglar Gülçehre, KyungHyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *NIPS*, 2014.

[12] Jeffrey Dean, Greg S. Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, Marc'Aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, and Andrew Y. Ng. Large scale distributed deep networks. In *NIPS*, 2012.

[13] John Duchi, Michael I. Jordan, and Martin J. Wainwright. Privacy aware learning. *Journal of the Association for Computing Machinery*, 2014.

[14] Cynthia Dwork and Aaron Roth. *The Algorithmic Foundations of Differential Privacy*. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014.

[15] Olivier Fercoq, Zheng Qu, Peter Richtárik, and Martin Takác. Fast distributed coordinate descent for non-strongly convex losses. In *Machine Learning for Signal Processing (MLSP), 2014 IEEE International Workshop on*, 2014.

[16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. Book in preparation for MIT Press, 2016.

[17] Ian J. Goodfellow, Oriol Vinyals, and Andrew M. Saxe. Qualitatively characterizing neural network optimization problems. In *ICLR*, 2015.

[18] Slawomir Goryczka, Li Xiong, and Vaidy Sunderam. Secure multiparty aggregation with differential privacy: A comparative study. In *Proceedings of the Joint EDBT/ICDT 2013 Workshops*, 2013.

[19] Benjamin Graham. Fractional max-pooling. *CoRR*, abs/1412.6071, 2014. URL `http://arxiv.org/abs/1412.6071`.

[20] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8), November 1997.

[21] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[22] Yoon Kim, Yacine Jernite, David Sontag, and Alexander M. Rush. Character-aware neural language models. *CoRR*, abs/1508.06615, 2015.

[23] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtarik, Ananda Theertha Suresh, and Dave Bacon. Federated learning: Strategies for improving communication efficiency. In *NIPS Workshop on Private Multi-Party Machine Learning*, 2016.

[24] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, 2009.

[25] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*. 2012.

[26] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 1998.

[27] Chenxin Ma, Virginia Smith, Martin Jaggi, Michael I Jordan, Peter Richtárik, and Martin Takáč. Adding vs. averaging in distributed primal-dual optimization. In *ICML*, 2015.

[28] Ryan McDonald, Keith Hall, and Gideon Mann. Distributed training strategies for the structured perceptron. In *NAACL HLT*, 2010.

[29] Natalia Neverova, Christian Wolf, Griffin Lacey, Lex Fridman, Deepak Chandra, Brandon Barbello, and Graham W. Taylor. Learning human identity from motion patterns. *IEEE Access*, 4:1810–1820, 2016.

[30] Jacob Poushter. Smartphone ownership and internet usage continues to climb in emerging economies. Pew Research Center Report, 2016.

[31] Daniel Povey, Xiaohui Zhang, and Sanjeev Khudanpur. Parallel training of deep neural networks with natural gradient and parameter averaging. In *ICLR Workshop Track*, 2015.

[32] William Shakespeare. The Complete Works of William Shakespeare. Publically available at `https://www.gutenberg.org/ebooks/100`.

[33] Ohad Shamir and Nathan Srebro. Distributed stochastic optimization and learning. In *Communication, Control, and Computing (Allerton)*, 2014.

[34] Ohad Shamir, Nathan Srebro, and Tong Zhang. Communication efficient distributed optimization using an approximate newton-type method. *arXiv preprint arXiv:1312.7853*, 2013.

[35] Reza Shokri and Vitaly Shmatikov. Privacy-preserving deep learning. In *Proceedings of the 22Nd ACM SIGSAC Conference on Computer and Communications Security*, CCS '15, 2015.

[36] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. 15, 2014.

[37] Latanya Sweeney. Simple demographics often identify people uniquely. 2000.

[38] TensorFlow team. Tensorflow convolutional neural networks tutorial, 2016. `http://www.tensorflow.org/tutorials/deep_cnn`.

[39] White House Report. Consumer data privacy in a networked world: A framework for protecting privacy and promoting innovation in the global digital economy. *Journal of Privacy and Confidentiality*, 2013.

[40] Tianbao Yang. Trading computation for communication: Distributed stochastic dual coordinate ascent. In *Advances in Neural Information Processing Systems*, 2013.

[41] Ruiliang Zhang and James Kwok. Asynchronous distributed admm for consensus optimization. In *ICML*. JMLR Workshop and Conference Proceedings, 2014.

[42] Sixin Zhang, Anna E Choromanska, and Yann LeCun. Deep learning with elastic averaging sgd. In *NIPS*. 2015.

[43] Yuchen Zhang and Lin Xiao. Communication-efficient distributed optimization of self-concordant empirical loss. *arXiv preprint arXiv:1501.00263*, 2015.

[44] Yuchen Zhang, Martin J Wainwright, and John C Duchi. Communication-efficient algorithms for statistical optimization. In *NIPS*, 2012.

[45] Yuchen Zhang, John Duchi, Michael I Jordan, and Martin J Wainwright. Information-theoretic lower bounds for distributed statistical estimation with communication constraints. In *Advances in Neural Information Processing Systems*, 2013.

[46] Martin Zinkevich, Markus Weimer, Lihong Li, and Alex J. Smola. Parallelized stochastic gradient descent. In *NIPS*. 2010.

A Supplemental Figures and Tables

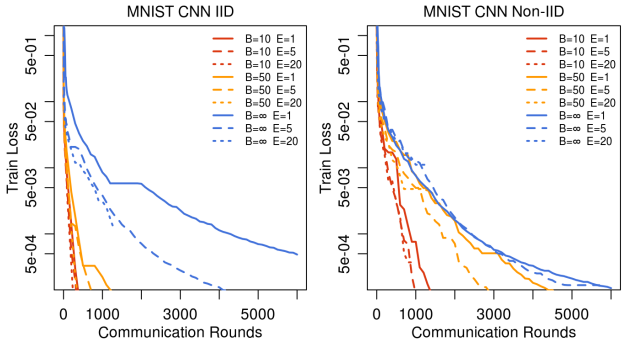

Figure 6: Training set convergence for the MNIST CNN. Note the y-axis is on a log scale, and the x-axis covers more training than Figure 2. These plots fix $C = 0.1$.

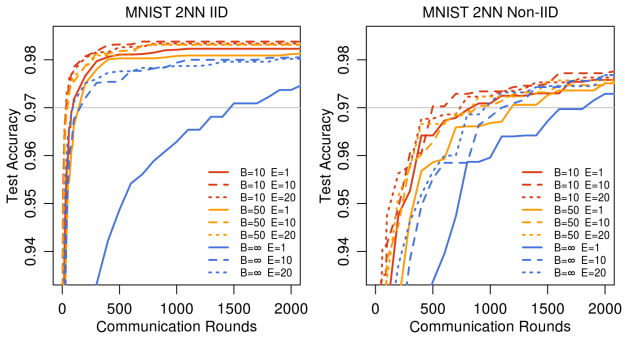

Figure 7: Test set accuracy vs. communication rounds for MNIST 2NN with $C = 0.1$ and optimized η. The left column is the IID dataset, and right is the pathological 2-digits-per-client non-IID data.

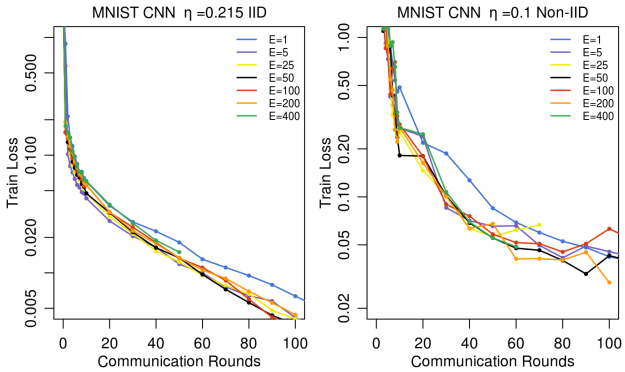

Figure 8: The effect of training for many local epochs (large E) between averaging steps, fixing $B = 10$ and $C = 0.1$. Training loss for the MNIST CNN. Note different learning rates and y-axis scales are used due to the difficulty of our pathological non-IID MNIST dataset.

Table 4: Speedups in the number of communication rounds to reach a target accuracy of 97% for `FedAvg`, versus `FedSGD` (first row) on the MNIST 2NN model.

MNIST 2NN	E	B	u	IID	NON-IID
FEDSGD	1	∞	1	1468	1817
FEDAVG	10	∞	10	156 (9.4×)	1100 (1.7×)
FEDAVG	1	50	12	144 (10.2×)	1183 (1.5×)
FEDAVG	20	∞	20	92 (16.0×)	957 (1.9×)
FEDAVG	1	10	60	92 (16.0×)	831 (2.2×)
FEDAVG	10	50	120	45 (32.6×)	881 (2.1×)
FEDAVG	20	50	240	39 (37.6×)	835 (2.2×)
FEDAVG	10	10	600	34 (43.2×)	497 (3.7×)
FEDAVG	20	10	1200	32 (45.9×)	738 (2.5×)

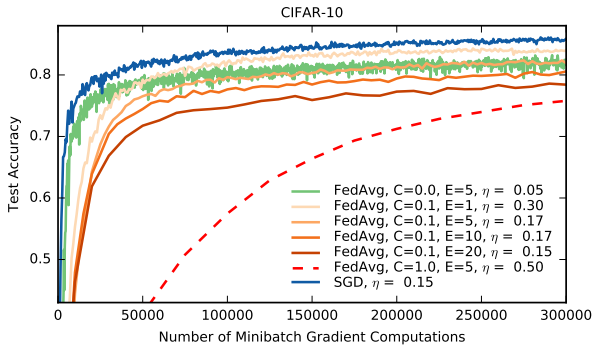

Figure 9: Test accuracy versus number of minibatch gradient computations ($B = 50$). The baseline is standard sequential SGD, as compared to `FedAvg` with different client fractions C (recall $C = 0$ means one client per round), and different numbers of local epochs E.

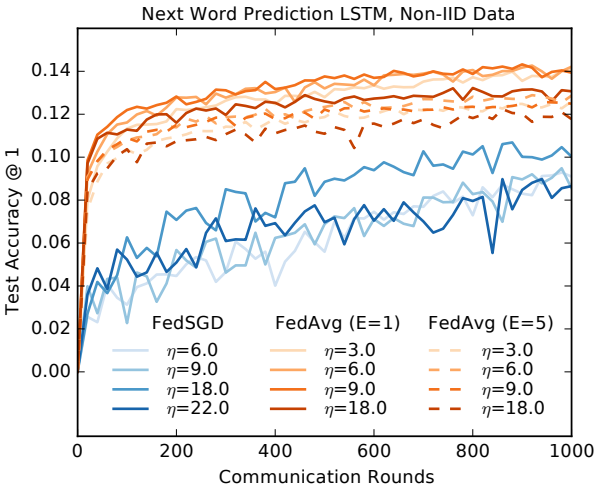

Figure 10: Learning curves for the large-scale language model word LSTM, with evaluation computed every 20 rounds. `FedAvg` actually performs better with fewer local epochs E (1 vs 5), and also has lower variance in accuracy across evaluation rounds compared to `FedSGD`.